

Mini-Lecture 6 - Orchestration Fundamentals (with Kubernetes)

II.2421 – Cloud & Security

Isep

Spring 2026

Where this fits in the course

- **Lab 5:** containers as *workload packaging* + *supply chain boundary*
- **Today:** orchestration as *system behavior at scale* → **Lab 6:** Kubernetes as a *control plane* enforcing desired state and policy

Shift in focus:

from “*what runs inside one container*” → “*how many run, where, and under which rules*”

deploy → expose → scale → authorize → segment

Why orchestration exists

Single host (Lab 5 world)

- Start container manually
- Restart if it crashes
- Fixed network endpoint

Distributed reality

- Many containers across many nodes
- Failures are normal (crash, node loss)
- Instances scale up/down dynamically
- IPs and locations change

Problem:

How do we keep the system behaving correctly despite change?

Answer:

Orchestration = automated control of distributed workloads

Orchestration in one sentence

Orchestration continuously reconciles desired state to actual state across distributed workloads

- You declare **intent** (*e.g., “run 3 replicas of this image and expose it”*)
- A system compares **desired** vs **current**
- It acts continuously to close the gap

Kubernetes = one implementation of this model
based on a **control plane** and **API objects**

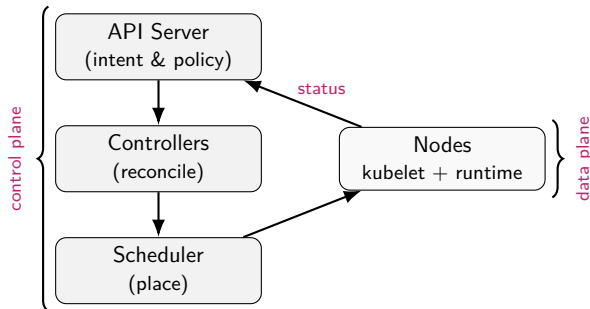
Architecture: control plane vs nodes (conceptual)

- **Control plane (decide + enforce intent):**

- **API server:** front door and policy choke point
- **Controllers:** reconciliation logic
- **Scheduler:** placement decision

- **Nodes (execute):**

- kubelet + runtime run pods
- report status back to the control plane



Minimal objects for today: Deployment and Service

Pod (unit of execution)

- smallest schedulable unit
- ephemeral by design
- created/replaced by controllers

Deployment (desired state)

- desired replicas
- rollout/replace policy
- repairs drift by creating pods

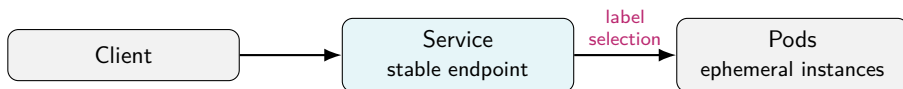
Service (exposure boundary)

- stable name/IP
- selects pods via labels
- decouples clients from pod lifecycle changes

Deployment defines *what/how many*
Service defines *how to reach it reliably*

Expose: Service makes reachability explicit

- A Service creates a **stable access point** over changing pods
- "Expose" is a **boundary decision**: who can reach the workload and from where
- It is not a load balancer first — it is a **naming and discovery mechanism**



Exposure ladder (concept):
ClusterIP (internal) → NodePort (node boundary) → Ingress (edge boundary)

Scale is reconciliation (desired vs current)

- When you change `replicas`, you are not starting processes directly
- You update **desired state**; controllers act until **current state matches**
- This separation is powerful *and risky*: **bad intent scales too**
- The same mechanism handles **failures**: missing pods are recreated automatically

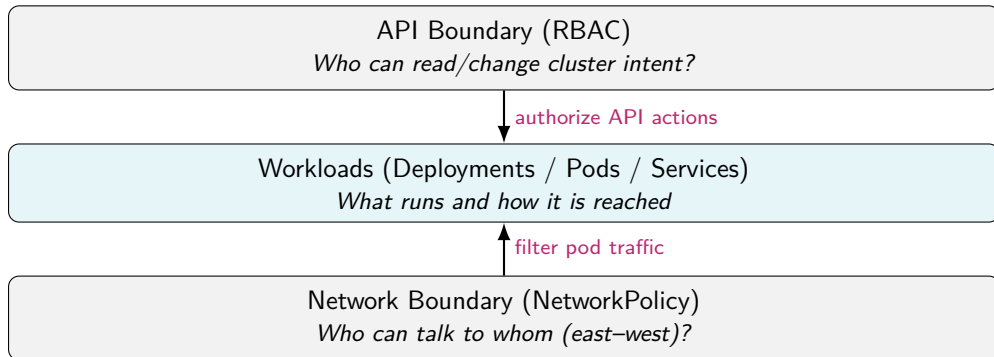
Architectural lens: scaling is a change to intent that triggers a control loop



| If the control plane accepts bad intent, Kubernetes will faithfully enforce it at scale

Policy surfaces: who can change intent vs who can communicate

RBAC and NetworkPolicy protect different boundaries



RBAC: the API boundary (control plane authorization)

- RBAC answers: **who can do what** to **which API objects** in **which scope**
- It protects **intent and configuration**: deployments, services, secrets, policies

Lab proof: same cluster, different identity $\Rightarrow$ **allowed** vs **forbidden**

NetworkPolicy: the runtime network boundary (east–west segmentation)

- NetworkPolicy answers: **which pods can talk to which pods** on which ports/protocols
- Typical posture: **default deny** + explicit allow for known paths
- **Implementation dependency:** enforcement depends on the CNI plugin

Lab proof: deny $\Rightarrow$ request fails; allow $\Rightarrow$ request succeeds

CNI: Container Network Interface: plugin architecture for Kubernetes network implementation. Not all CNIs support NetworkPolicy enforcement.

Five statements to remember and prove in the lab

- ① Kubernetes reconciles **desired state** to **actual state**
- ② The API server is the **front door** and policy choke point: everything is an API object
- ③ Deployment + Service are **contracts**: intent + stable reachability
- ④ RBAC protects **who can change intent**; NetworkPolicy protects **who can talk to whom**
- ⑤ Managed Kubernetes shifts responsibilities, not accountability: you still own **config, identity, network boundaries, images**

Next: you will prove each statement in the lab with an observation or a denial